

NIFTY Option Offset Scalper

System Architecture & Software Design Document (v2.0)

Author: Solution Architect / Technical Lead

Version: 2.0

Architecture: Event Driven (Without Redis/BullMQ)

Language: Node.js (Express)

Database: MySQL 8

Broker: Kotak Neo API

Market Data: Kotak Neo WebSocket

1. Objective

Build a **high-speed automated option selling engine** for **NIFTY 50** capable of:

- Auto Expiry Selection
 - Auto Strike Selection
 - CE / PE / Both Legs
 - Offset Based Selling
 - ₹1 Premium Capture
 - Dynamic Strike Selection
 - Automatic Re-Quote
 - Automatic Risk Management
 - Dynamic Stop Loss
 - Daily Risk Controls
 - Web Dashboard
 - Complete Audit Logs
-

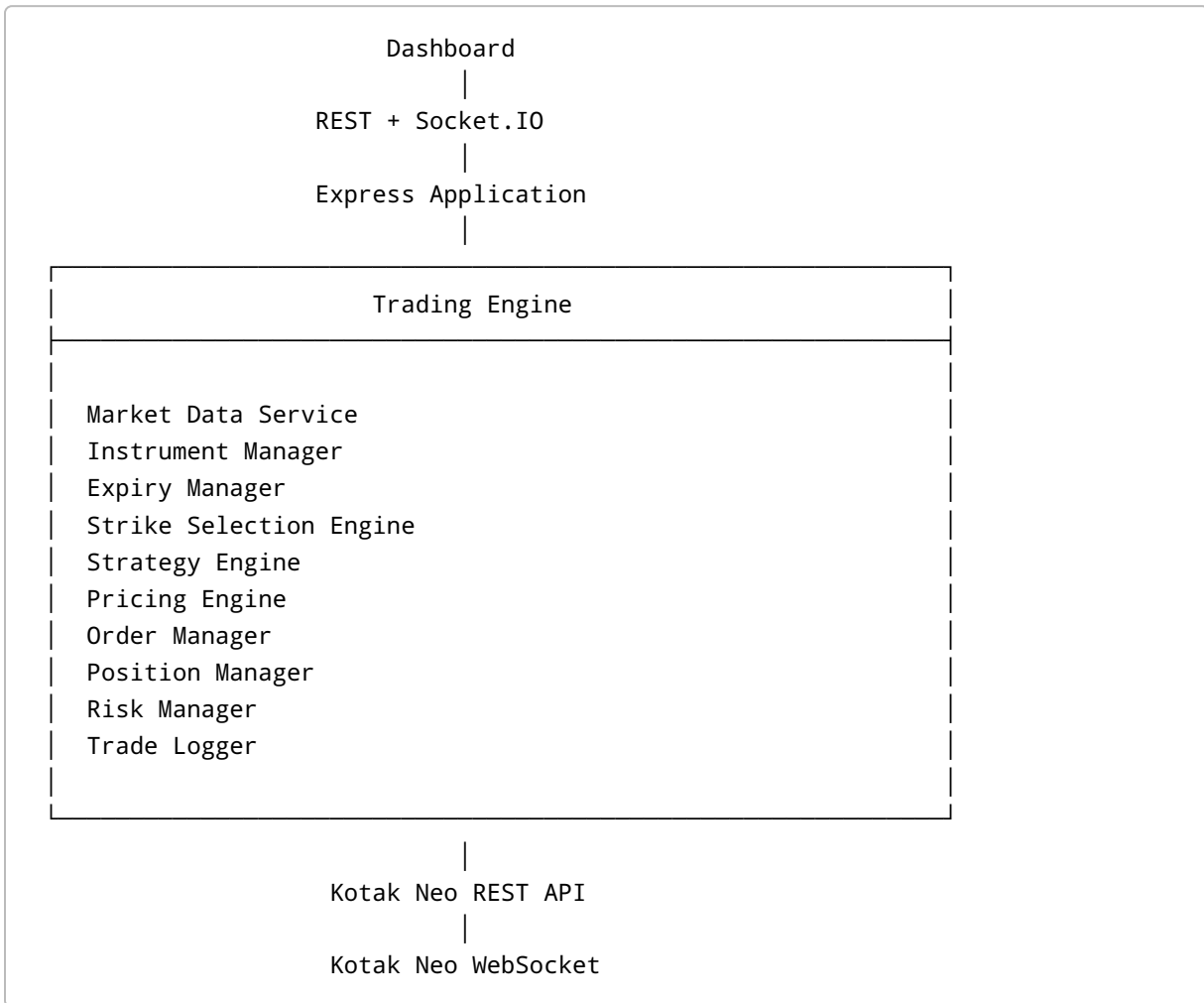
2. Design Principles

The engine must

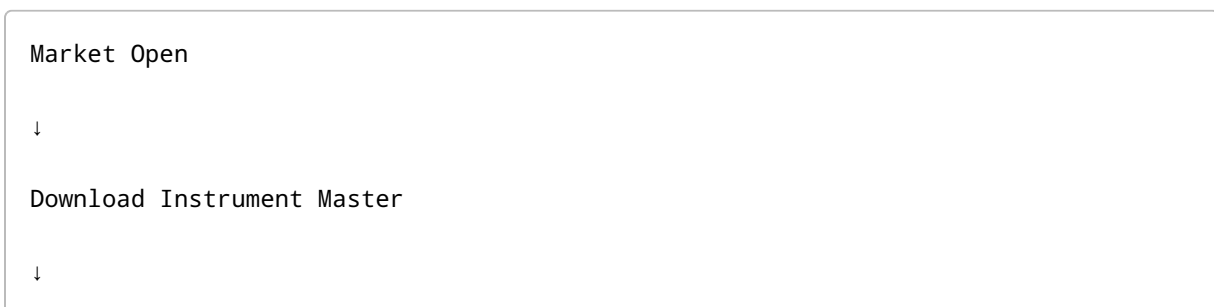
- Never chase the market
- Never average losses
- Never open duplicate positions
- Never leave stale pending orders
- Never trade without liquidity

- Never continue after daily loss limit

3. Overall Architecture



4. Trading Cycle



Select Expiry

↓

Select Strike

↓

Validate Premium

↓

Apply Offset

↓

Place SELL Limit

↓

Filled?

└─ No

| Cancel

| Recalculate

| Place Again

|

└─ Yes

↓

Create BUY Target

↓

Target Hit ?

└─ Yes

|

Book Profit

↓

Repeat

└ No

↓

SL Hit ?

↓

Exit

↓

Cooldown

↓

Repeat

5. Supported Market

NIFTY

Future Expansion

BANKNIFTY

FINNIFTY

MIDCPNIFTY

6. Expiry Selection

Modes

CURRENT WEEKLY

NEXT WEEKLY

MONTHLY

MANUAL

Configuration

```
{  
  "expiryMode": "CURRENT_WEEKLY"  
}
```

7. Strike Selection

Mode 1

ATM

NIFTY

25120

↓

ATM

25150

Mode 2

ATM Offset

ATM

25150

Offset

2

↓

CE

25250

↓

PE

25050

Mode 3

Premium Based

Instead of fixed strikes

Bot searches premium.

Example

Target Premium

₹12

Tolerance

±2

Valid

₹10

↓

₹14

If no strike found

No Trade

8. Dynamic Strike Lock

Once a strike is selected

25150 CE

25150 PE

Bot NEVER changes strike until

Trade Closed

This prevents

- Chasing market
- Selling lower strikes continuously
- Reversing losses due to moving premium

9. Premium Validation

Before every order

Read Live Premium

↓

Inside Range ?

↓

YES

↓

Place Order

↓

NO

↓

Skip

10. Offset Engine

Configuration

Offset

0.20

0.50

1

1.50

2

Formula

Sell Price

=

Ask

+

Offset

Example

Ask

12.40

↓

Offset

1

↓

Sell

13.40

11. Order Management

Pending

Pending > 10 Seconds

↓

Cancel

↓

Recalculate

↓

Place Again

Filled

Immediately create

BUY Target

Example

Sell

13.40

↓

Buy

12.40

12. Target Engine

Target

1 Point

Configurable

0.50

1

1.50

2

13. Stop Loss Engine

Fixed

Entry

13.40

↓

SL

15.40

Percentage

Entry

13.40

↓

20%

↓

16.08

Premium Increase

Premium Increase

>2

↓

Exit

Time Stop

Maximum Hold

60 Seconds

↓

Exit

14. Market Movement Protection

If NIFTY moves

40 Points

Within

30 Seconds

Then

Pause New Entry

Existing Positions

Continue Managing

15. Position Rules

Maximum

1 CE

1 PE

Never

2 CE

2 PE

No averaging

No martingale

No pyramid

16. Daily Risk

Daily Profit

₹5000

↓

Stop Trading

Daily Loss

₹3000

↓

Square Off

↓

Disable Strategy

Consecutive Loss

3

↓

Cooldown

15 Minutes

17. Position Timeout

Trade Open

60 Seconds

↓

Exit

No exceptions

18. Configuration

```
{
  "symbol": "NIFTY",
  "expiryMode": "CURRENT_WEEKLY",
  "tradeMode": "BOTH",
  "strikeMode": "PREMIUM",
  "targetPremium": 12,
  "premiumTolerance": 2,
  "atmOffset": 2,
  "sellOffset": 1,
  "target": 1,
  "stopLoss": 2,
  "positionTimeout": 60,
  "pendingTimeout": 10,
  "maxOpenCE": 1,
  "maxOpenPE": 1,
  "marketMovePause": 40,
  "cooldownAfterSL": 300,
  "maxDailyLoss": 3000,
  "maxDailyProfit": 5000,
  "maxConsecutiveLoss": 3
}
```

19. Database Tables

strategy_settings

```
id
symbol
expiry_mode
strike_mode
```

```
target_premium  
premium_tolerance  
atm_offset  
sell_offset  
target  
stoploss  
position_timeout
```

instruments

```
token  
symbol  
expiry  
strike  
option_type  
lot_size
```

orders

```
id  
broker_order_id  
token  
symbol  
side
```

```
price
qty
status
filled_price
created_at
updated_at
```

positions

```
id
symbol
entry
exit
target
sl
qty
profit
status
```

pnl

```
daily
weekly
monthly
realized
```

unrealized

trade_logs

id
event
request
response
latency
created_at

20. API

POST /start
POST /stop
POST /pause
GET /status
GET /settings
POST /settings
GET /orders
GET /positions
GET /logs
GET /pnl

21. Dashboard

Widgets

Live NIFTY
Current Expiry
ATM Strike
Selected Strike
Current Premium
Pending Orders
Filled Orders
Open Position
Today's P&L
Risk Status
API Latency
WebSocket Status

22. Logging

Every event logged

Tick
Scanner
Strike
Premium
Order

Modify

Cancel

Fill

Target

Stoploss

Error

Risk

23. Recovery

If API fails

Retry

↓

3 Attempts

↓

Backoff

↓

Stop

If WebSocket disconnects

Reconnect

↓

Resubscribe

↓

24. Security

- JWT Authentication
- HTTPS
- Environment Variables
- API Key Encryption
- IP Whitelist
- Audit Trail

25. Performance Goals

Item	Target
Tick Processing	<10 ms
Order Decision	<20 ms
API Response	<200 ms
Dashboard Update	<500 ms
Memory Usage	<500 MB
CPU	<30%

26. Future Enhancements

- Multi-account execution
- Multi-broker support
- Volatility-aware offset calculation
- Bid/ask imbalance filter
- Open interest change filter
- Greeks-based strike ranking
- Auto trading session scheduler
- Telegram/WhatsApp alerts
- AI-based premium prediction
- Historical trade analytics
- Paper trading mode
- Strategy backtesting engine

27. Development Phases

Phase 1

- Broker Login
- WebSocket
- Instrument Download
- Market Scanner

Phase 2

- Strike Selection
- Offset Engine
- Order Placement
- Position Management

Phase 3

- Target Engine
- Stop-Loss Engine
- Risk Controls
- Dashboard

Phase 4

- Analytics
- Reporting
- Notifications
- Performance Optimization

28. Key Engineering Rules

1. Lock the selected strike until the trade is closed.
2. Never open more than one CE and one PE position simultaneously.
3. Revalidate premium immediately before placing every order.
4. Cancel and requote stale limit orders.
5. Pause new entries during rapid market movement.
6. Enforce daily P&L and consecutive-loss limits.
7. Maintain complete audit logs for every state transition.
8. Build all modules as independent services with clear interfaces to simplify testing and future enhancements.